Carlos Cotrini

# Building ML AI Applications

## HNSW: Hierarchical Navigable Small World Graphs

Spring 2026

# 1. Nearest Neighbor Search

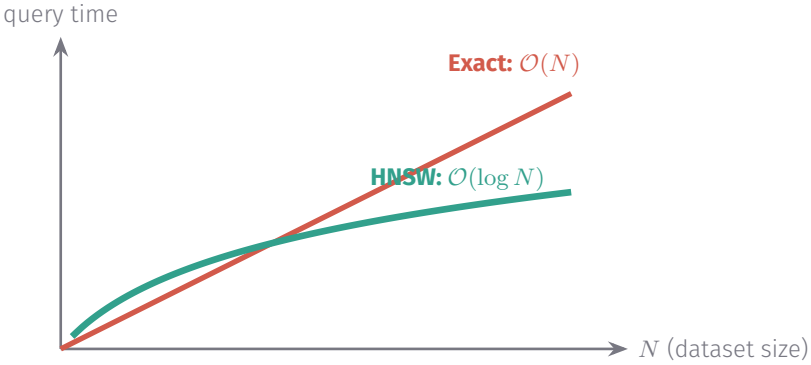
Find the $K$ closest points to a query, fast.

# The Task

- $N$ data points
- Query $t$
- Find $K$ nearest

**Exact:** scan all $N$
$\Rightarrow \mathcal{O}(N)$ per query

# Goal: Sublinear Search



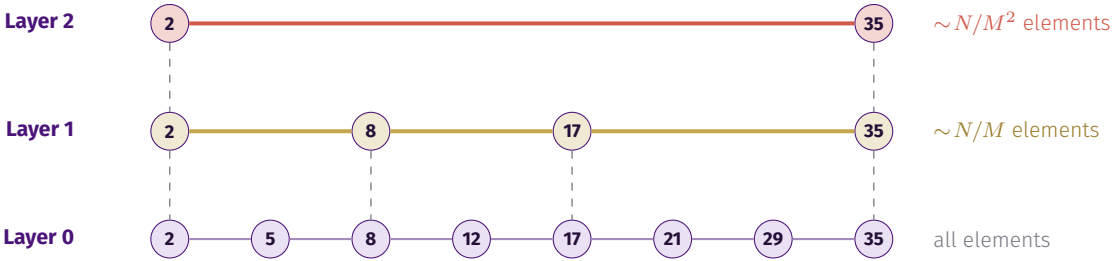

**Key idea**

Build a **multi-layer graph** that enables **greedy navigation** to nearest neighbors.

# 2. The Skip-List Analogy

HNSW is a skip list generalized to metric spaces.

# Skip Lists: Express Lanes



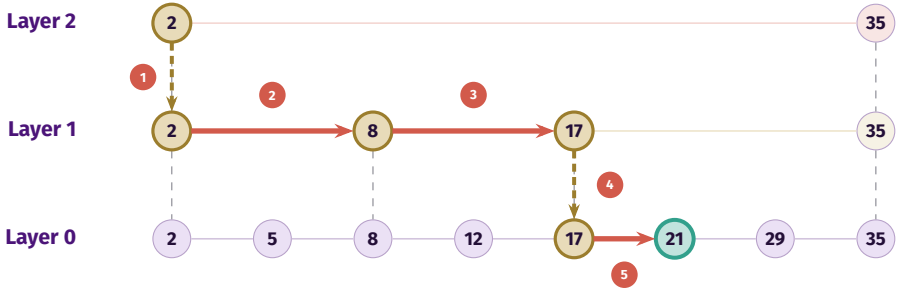

**Search strategy:**

1. Start at top layer
2. Move right while possible
3. Drop down, repeat

**Complexity:**

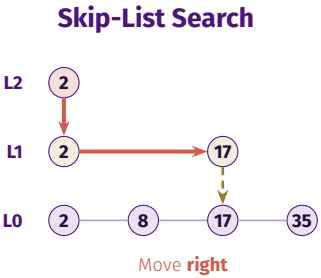
- $\mathcal{O}(\log N)$ expected
- Upper layers = express lanes

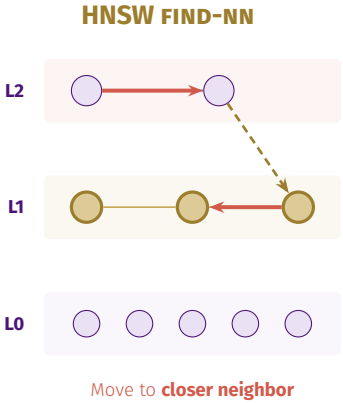
# Skip-List Search: Finding 21




**5 comparisons** vs. 6 for linear scan. Express lanes skip most elements.

# Skip-List Search $\longleftrightarrow$ HNSW FIND-NN



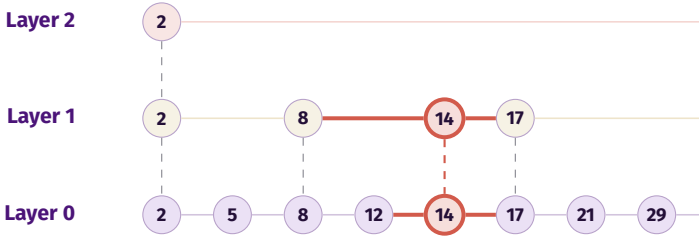




Same strategy: start at top layer, greedily advance, drop when stuck.
**Skip list:** "next in sorted order" $\longleftrightarrow$ **HNSW:** "closest in metric distance"

# Skip-List Insert: Adding 14

**Three steps:**

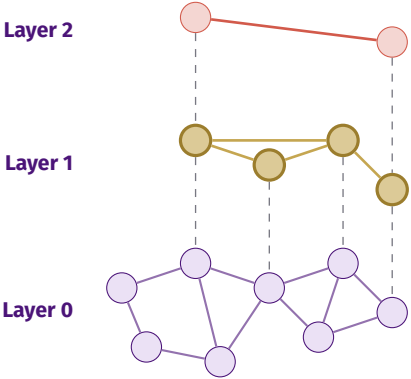
1. **Search** for position
   $\rightarrow$ between 12 and 17
2. **Coin flips** for layer
   H, T $\Rightarrow \ell_{14} = 1$
3. **Splice** into lists
   L0: $12 \rightarrow \mathbf{14} \rightarrow 17$
   L1: $8 \rightarrow \mathbf{14} \rightarrow 17$




**Analogy to HNSW INSERT:** search $\rightarrow$ BEAM-SEARCH, coin flips $\rightarrow \mathrm{Bern}(1/M)$, splice $\rightarrow$ wire edges via SEL-NEIGHBORS

# From 1D to Metric Spaces

| **Skip List** | **HNSW** |
|---|---|
| Sorted linked list | Proximity graph |
| Left/right neighbors | $M$ nearest neighbors |
| Promotion prob. $\frac{1}{2}$ | Promotion prob. $\frac{1}{M}$ |
| "Move right" | "Move closer" |
| Drop to next layer | Drop to next layer |

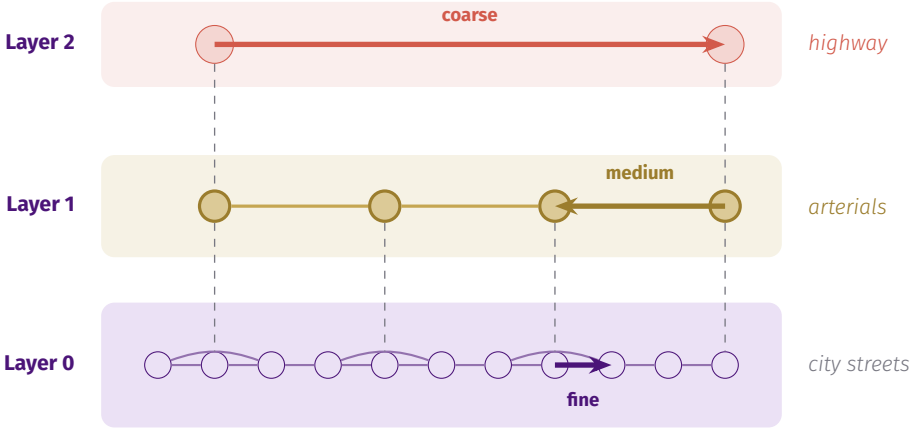



In 1D with $M=2$, HNSW *is* a skip list. In higher dimensions, proximity graphs replace sorted linked lists.

# 3. Why Layers?

Separating long-range and short-range links.

# Layered Hierarchy




Each layer has bounded degree $\leq M$ $\Rightarrow$ no hubs, no bottlenecks.

# 4. Data Structure

How elements are organized across layers.

# Layer Assignment: Coin Flips

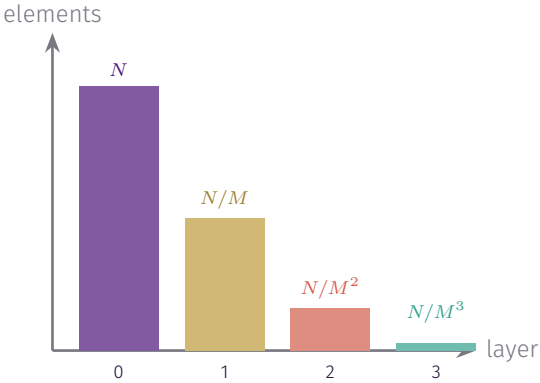
Flip a biased coin (heads prob. $\frac{1}{M}$).

Count heads before first tail:

$$\ell_w = 0, 1, 2, \ldots$$

**Geometric distribution:**

$$\Pr[\ell_w \geq k] = \left(\frac{1}{M}\right)^k$$




$M$=16, $N$=$10^6$:

Layer 2 ≈ 4000 elements

Typically 4–6 layers total

# Key Parameters

| Parameter | Typical | Role |
|---|---|---|
| $M$ | 5–48 | Max neighbors per layer |
| $M_{\max 0} = 2M$ | 10–96 | Max neighbors on layer 0 |
| $ef_{\text{Constr}}$ | 100–500 | Beam width during build |
| $ef$ | $\geq K$ | Beam width at query time |

**Build-time (fixed)**

$M$ and $ef_{\text{Constr}}$ are baked into the index. Cannot be changed without rebuilding.

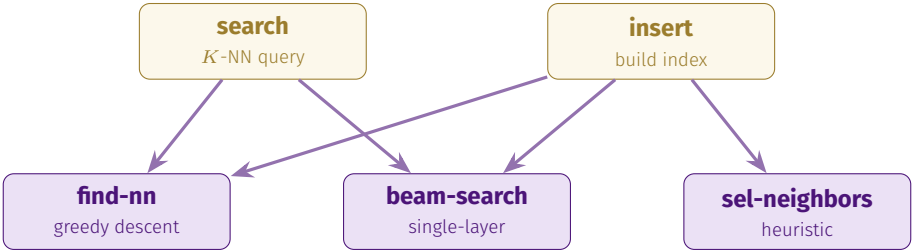
**Query-time (tunable)**

$ef$ trades speed for recall.
Increase $ef \Rightarrow$ higher recall, slower queries.

# 5. The Algorithms

Five building blocks of HNSW.

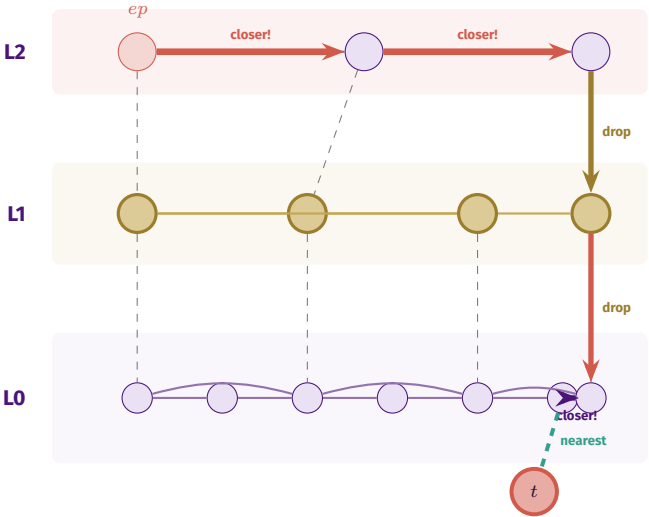
# Five Algorithms



# FIND-NN: Greedy Descent

1. Start at entry point, top layer
2. Move to closest neighbor
3. No improvement ⇒ drop one layer
4. Repeat until target layer

**No beam.** Single current node. Fast but can hit local minima.

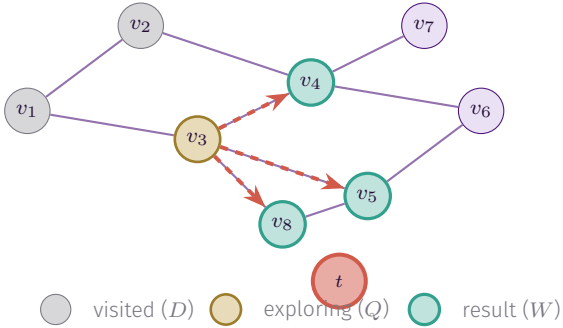


# BEAM-SEARCH: Single-Layer Exploration

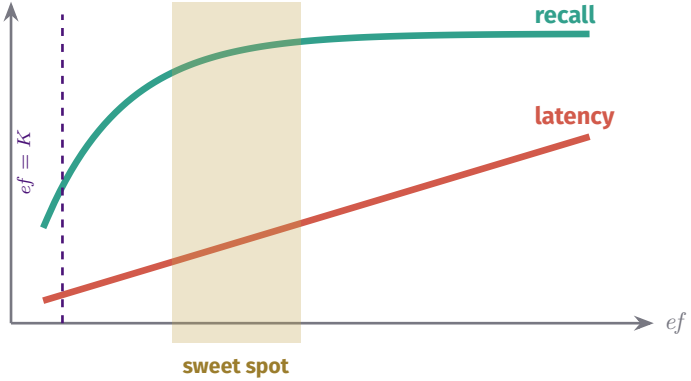
Three data structures:

- $Q$ candidates (min-heap)
- $W$ best $ef$ results (max-heap)
- $D$ visited (hash set)

**Stop** when best candidate > worst result.



# BEAM-SEARCH: The $ef$ Knob




$ef = K$: fastest, lowest recall

$ef = 200$, $K = 10$: examines 200 candidates, returns top 10

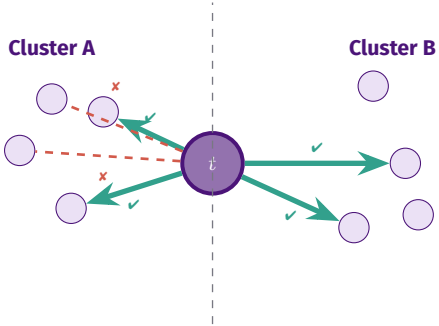
# SEL-NEIGHBORS: Directional Diversity

**Naïve:** pick $M$ closest
$\Rightarrow$ all neighbors in one cluster!

**Heuristic:** add $w^*$ only if

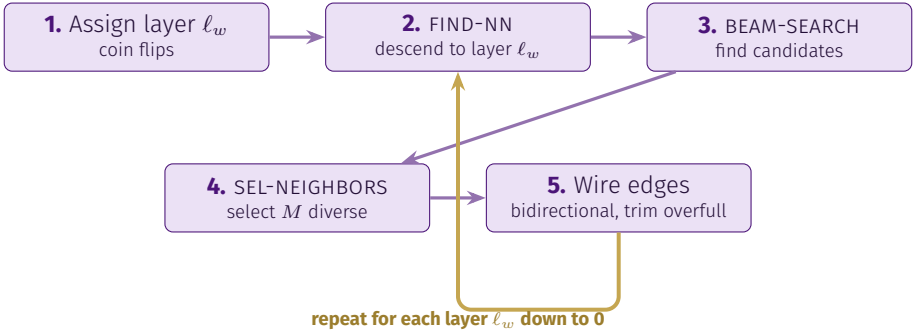
$$d(w^*, t) < \min_{v \in N} d(w^*, v)$$

$\Rightarrow$ angular spread $\approx$ Delaunay




Rejects candidates "overshadowed" by already-chosen neighbors. Ensures connectivity across cluster boundaries.

# INSERT: Building the Index




**Key detail**

Inserting $w$ may push existing node $v$ over its degree limit.
We trim $v$'s neighbors with SEL-NEIGHBORS, not simple truncation.

# SEARCH: Answering $K$-NN Queries

1. **Greedy descent**
   layers $L \to 1$: cheap, no beam

2. **Beam search on layer 0**
   find $ef$ candidates among all $N$ elements

3. **Return top $K$**

Layer 0 contains *all* $N$ elements: the true nearest neighbor is always there.
Upper layers are just a fast way to find a good starting point.

# 6. Summary

# HNSW in One Slide

| Property | Value |
|---|---|
| Analogy | Skip list |
| Core idea | Layered links |
| Search | $\mathcal{O}(\log N)$ |
| Build | $\mathcal{O}(N \log N)$ |
| Memory | $\sim 8M$ B/elem |
| Key insight | SEL-NEIGHBORS |
| Knob | $ef$ (query) |

Three reinforcing design choices:

1. **Layered hierarchy**
   coarse $\rightarrow$ fine navigation
2. **Heuristic selection**
   connectivity across clusters
3. **Bounded degree**
   no hub bottlenecks

HNSW = skip-list structure + diverse neighbor selection + bounded degree at every layer.